ReproMut: A Controlled Mutation Benchmark for Comparing Reproducibility Auditors on Small Machine-Learning Experiments

Preprint — work in progress (draft v0.2)

Hamjeth Misree
Independent researcher
hamjeth68@gmail.com
ORCID: 0009-0008-2548-6920

8 November 2026

Status of this document. This is an in-progress preprint describing a benchmark design and an *exploratory development run*. The hypothesis thresholds were written down before the run but were not publicly preregistered; all results should be read as exploratory. A frozen, preregistered confirmatory run is planned for v0.1.0 (Section 10). **New in v0.2:** a *pre-registered* external validation (E1) on independently written code (Section 7).

Abstract

Computational reproducibility failures in machine-learning (ML) experiments often stem from small, mundane causes: unfixed random seeds, silently drifting dependency defaults, hidden configuration overrides, stale cached artifacts and inconsistent preprocessing. Many checks exist to guard against these causes, but there is little controlled evidence about *which* kinds of check catch *which* kinds of failure, and at what cost. We present ReproMut, an executable benchmark that injects predeclared reproducibility mutations into small, CPU-only, fully synthetic ML experiments with known ground truth. Version 0.1 comprises 28 baseline experiments (binary and multiclass classification, regression, clustering), 12 result-changing mutation operators and 7 semantics-preserving control operators, yielding 452 cases. Every declared label and divergence stage is confirmed by an independent tolerance-based execution oracle. We compare five auditing strategies: no audit, a seed-only check, a static manifest comparison, a dynamic auditor that re-executes the package and compares per-stage fingerprints, and a static–dynamic combination. In an exploratory development run, the seed-only check detected 29.4% of result-changing mutations (95% CI 24.3–35.1%) and the static manifest 72.8% (67.2–77.7%), while the dynamic auditor detected all 272 (98.6–100%) with no false positives on 180 controls. The static manifest detected none of the 74 mutations that live outside the declared configuration (stale caches, environment-file overrides, dependency drift) and raised false alarms on 12 harmless thread-count changes. Because the dynamic auditor and the oracle both execute the package, the dynamic result is largely expected by construction; the substantive findings concern the blind spots and false alarms of cheaper checks, localisation, and cost. During construction the oracle also exposed three non-obvious invariances in which an apparently disruptive change leaves results unchanged. In a pre-registered external validation on 30 randomly sampled, independently written scikit-learn example scripts (199 mutation cases), the seed-only check again missed every non-seed change (recall 71–75%), the static manifest raised false alarms on the 13% of plausible "breaking" changes that turned out not to alter results (false-positive rate 6.5–7.7%), and 2 of 32 runnable scripts were not reproducible run-to-run before any mutation. Code, data generators, tests and raw results are released openly.

1 Introduction

Reproducing a published computational result, i.e., re-running the authors' code and data to obtain the same numbers, is the weakest form of verification, yet it frequently fails. Surveys of AI research report that much of the information needed to reproduce empirical results is not documented [4]; large studies of research code find that a majority of shared scripts or notebooks do not execute cleanly or reproduce their outputs [10, 13]; and ML results vary substantially with sources of randomness and tooling that are rarely controlled [1, 16]. Community responses include checklists and code-submission policies [11], environment packaging tools [2], and seed and determinism guidance [3].

These safeguards embody different assumptions about where failures come from. A seed check assumes randomness is the main threat. A static manifest (a hash of declared configuration and dependency specifications) assumes that what matters is written down. A dynamic check, which actually re-executes the experiment, makes no such assumption but costs compute. We are not aware of a controlled comparison that measures, against known ground truth, what each class of check misses, how often it raises false alarms on harmless changes, and whether it can tell the user *where* in the pipeline the divergence began.

Contributions. This preprint contributes:

1. **A controlled benchmark.** 28 small synthetic ML experiments packaged in a simple executable format, 12 result-changing mutation operators spanning random state, data order, data generation, numeric precision, dependency drift, hidden configuration, caching and train/evaluation mismatch, and 7 semantics-preserving controls (Section 3).

2. **Oracle-verified ground truth.** An execution oracle, independent of the auditors, that confirms every declared label and first-divergence stage across all 452 cases (Section 3.4).

3. **A comparison of five auditing strategies** on detection, false-positive rate, stage localisation and runtime (Sections 4–6).

4. **Three empirically discovered invariances** in which a plausible "breaking" change does not alter results, motivating harder controls for future auditors (Section 6.4).

5. **A pre-registered external validation** on independently written ML scripts, where ground truth is decided by execution rather than declared (Section 7).

All artifacts are released under open licences (Section 11).

2 Related work

Reproducibility of ML research. Gundersen and Kjensmo [4] assessed 400 AI papers and found that none documented every variable their metrics tracked. The NeurIPS 2019 reproducibility program introduced a code policy, a reproducibility challenge and a checklist [11]. Bouthillier et al. [1] showed that data sampling, initialisation and hyperparameter choices produce variance large enough to change benchmark conclusions, and Zhuang et al. [16] characterised how software and hardware tooling alone introduces nondeterminism in neural-network training. Kapoor and Narayanan [6] documented data leakage across many ML-based scientific fields. These works motivate controlling and documenting experimental choices; ReproMut instead asks how well automated checks enforce such control.

Executability of research code. Large-scale studies of Jupyter notebooks [10] and of R code in replication datasets [13] found widespread execution failures. Packaging tools such as ReproZip [2] capture an execution environment to ease re-execution. ReproMut is complementary: it assumes the package runs and evaluates whether it reproduces the *reported result*.

Mutation and metamorphic testing. Mutation analysis evaluates a test suite by injecting faults and measuring how many are detected [5]. It has been applied to ML implementations via metamorphic testing [15] and to deep-learning systems through source- and model-level mutation operators [7]. Dutta et al. [3] studied how ML projects use fixed seeds in tests. These works target the correctness of ML code or models; ReproMut targets the reproducibility of an experiment's reported result.

Closest work. Most closely related is the concurrent study by Shulepov [12], which applies four classes of reproducibility-relevant mutation (random seed, dependency pin, data split, cross-validation fold count) to real ML research repositories and evaluates whether the repositories' *existing* validation workflows detect them, reporting detection of 2 of 23 confirmed non-equivalent mutations. That study offers ecological validity on real code; ReproMut offers the complementary controlled setting: synthetic experiments with oracle-confirmed ground truth, a larger operator set that includes mutations invisible to configuration files, explicit semantics-preserving controls that allow false-positive rates to be measured, and a head-to-head comparison of auditor *classes* including stage localisation and cost. We regard the two approaches as mutually reinforcing and discuss the relationship further in Section 8.

3 Benchmark design

3.1 Experiment packages and pipeline stages

An experiment package is a directory containing a declared configuration (`experiment.json`), a resolved runtime environment (`.runtime_env.json`), an optional `.env` file and an optional preprocessing cache. A hermetic runner executes five stages and records their artifacts: *data* (generated features and targets), *split* (train/test indices), *preprocess* (transformed train and test matrices), *model* (predicted probabilities, predictions or cluster labels and objective) and *metrics*. The runner never reads the real process environment.

Two realistic failure patterns are built into the runner on purpose. First, keys of the form `REPROMUT_OVERRIDE__<path>` in a shipped `.env` file silently override configuration values. Second, when `preprocess.use_cache` is true, a shipped cache file is reused without validating its key. Dependency drift is modelled with a small stand-in library, `simlib`, whose default feature scaler changes from standard to robust scaling at version 2.0.0; packages declare the unpinned requirement `simlib>=1.0`.

3.2 Baseline experiments

Table 1 lists the 28 baseline experiments. All data are generated with scikit-learn [9] generators (400 samples), with per-feature random rescaling so that preprocessing matters. Every seed is derived as the first four bytes of SHA256("20261108/⟨experiment⟩/⟨role⟩"), so any seed can be recomputed from the experiment name. All 28 baselines were verified to be bit-for-bit deterministic across repeated runs.

Table 1: Baseline experiments (28). Each model is run on two data variants.

Task	Models	Variants	n
Binary classification	logistic regression, random forest, k-NN (distance), Gaussian NB	balanced / imbalanced	8
Multiclass (3 classes)	same as above	balanced / imbalanced	8
Regression	ridge, random forest, k-NN (distance), SGD	low / high noise	8
Clustering (3 blobs)	k-means, Gaussian mixture	separated / overlapping	4

Table 2: Mutation operators in v0.1. Hidden operators do not change the declared configuration.

Operator	Family	Expected stage	Applies to	Cases
Result-changing (positive)				
seed_unset	random-state	data	all	28
seed_time_derived	random-state	data	all	28
split_seed_changed	random-state	split	supervised	24
row_order_changed	data-order	data	regression	8
dependency_default_drift†	dependency	preprocess	scale-sensitive models	18
env_override†	hidden-config	model	all	28
dtype_float32	numeric	data	non-tree models	22
stale_cache†	caching	preprocess	all	28
train_eval_mismatch	config-mismatch	preprocess	supervised	24
label_noise_changed	data-generation	data	all	28
feature_generation_changed	data-generation	data	all	28
solver_tolerance_loosened	numeric	model	LR, SGD, k-means	8
Semantics-preserving (control)				
5 cosmetic (name, path, notes, log level, key order)	cosmetic	—	all	5×28
dependency_patch_bump	dependency	—	all	28
thread_count_changed	parallelism	—	RF, k-NN	12
Total				452

† Hidden operator.

3.3 Mutation operators

Table 2 lists the operators. *Positive* operators are declared result-changing; *control* operators are declared semantics-preserving. Each positive operator also declares the stage at which its effect should first appear. Operators apply only where meaningful (e.g., split-seed changes only to supervised tasks). Three positive operators are *hidden*: they leave the declared configuration untouched and act through the runtime environment, the `.env` file or the cache.

3.4 Ground-truth oracle

The oracle is deliberately independent of the auditors. It executes each mutated package three times and compares *full arrays*, not hashes, against the baseline run and against each other, using `atol` = `rtol` = 10^{-9}. A case is result-changing if the model or metrics stage diverges; the first divergent stage is recorded. The test suite requires that, for every one of the 452 cases, the oracle's observation equals the declared label, and for positives that the first divergent stage equals the declared stage.

4 Auditing strategies

Each auditor receives the shipped package and the *record* published with the original run: per-section hashes of the declared configuration, the declared seeds, SHA-256 fingerprints of the five stages, and

Table 3: Detection results (exploratory development run). 272 positive cases, 180 controls. CIs are Wilson 95%.

Method	TP	Recall (95% CI)	FP	FPR (95% CI)	F1	Median time
No audit	0	0.0% (0.0–1.4)	0	0.0% (0.0–2.1)	0.000	2.2 ms
Seed-only	80	29.4% (24.3–35.1)	0	0.0% (0.0–2.1)	0.455	0.06 ms
Static manifest	198	72.8% (67.2–77.7)	12	6.7% (3.9–11.3)	0.822	0.1 ms
Dynamic ($k = 2$)	272	100.0% (98.6–100)	0	0.0% (0.0–2.1)	1.000	4.6 ms
Combined	272	100.0% (98.6–100)	12	6.7% (3.9–11.3)	0.978	4.7 ms

the metrics. It outputs a flag ("will not reproduce") and a stage.

No audit. Executes the package once and reports success; never flags.

Seed-only. Flags any seed that is not an integer literal or that differs from the recorded seed.

Static manifest. Compares hashes of the declared configuration sections (task, seeds, data, split, preprocess, model, evaluation, dependencies); cosmetic fields are excluded. Does not execute.

Dynamic. Executes the package $k = 2$ times. Stage artifacts are fingerprinted after rounding floating-point values to 8 decimals and canonicalising -0.0 and NaN. Flags if the model or metrics fingerprint differs from the record or between runs; localises to the earliest stage that differs from the record, otherwise the earliest unstable stage.

Combined. Flags if either static or dynamic flags; uses the dynamic stage when available.

5 Protocol and statistics

We report counts and denominators for every method. Proportions carry Wilson 95% intervals [14]. Because all methods are evaluated on the same cases, comparisons are paired: we report the recall difference with a percentile bootstrap 95% interval (2,000 case resamples, seed 0) and an exact McNemar test on discordant pairs [8]. The development run used Python 3.10.12, NumPy 2.2.6, SciPy 1.15.3 and scikit-learn 1.7.2 on a 4-vCPU Linux aarch64 machine. A complete re-run produced identical case-level outcomes (all fields except timings). The test suite (658 tests) also passed in continuous integration on Ubuntu and macOS with Python 3.10 and 3.12.

The draft hypothesis, written before the run, was that the dynamic auditor would (i) detect at least 90% of positives, (ii) exceed seed-only recall by at least 30 percentage points and (iii) keep the false-positive rate below 5%.

6 Results

6.1 Detection and false alarms

Table 3 and Figure 1 summarise detection across the 272 positive and 180 control cases.

The seed-only check detected exactly the 80 random-state mutations and nothing else. The static manifest detected every mutation that alters the declared configuration (198 cases) but none of the 74 hidden mutations (28 stale-cache, 28 environment-override, 18 dependency-drift). Its 12 false positives were all thread-count controls: raising `n_jobs` for random forests and k-NN changes the declared configuration but not the result. The dynamic auditor detected all positives and raised

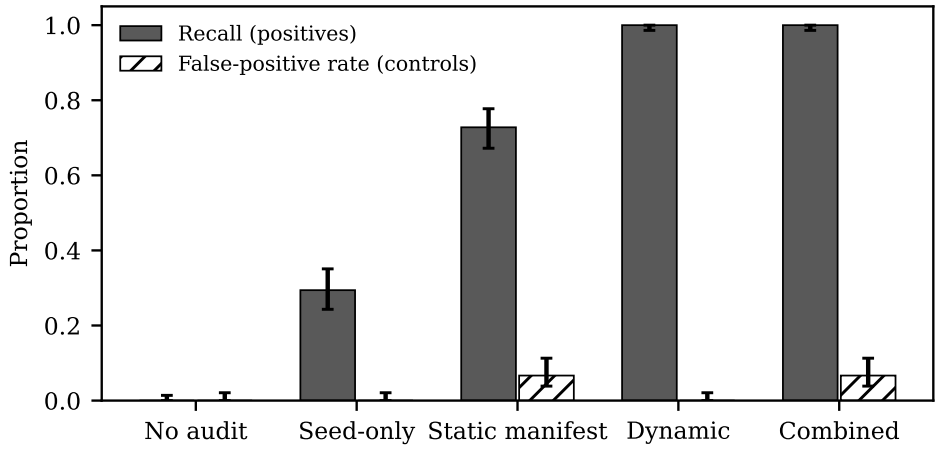


Figure 1: Recall on result-changing mutations and false-positive rate on controls, with Wilson 95% intervals.

no false alarms. Paired against seed-only, the dynamic auditor's recall was 70.6 percentage points higher (bootstrap 95% CI 65.1–76.1; 192 discordant cases, all favouring dynamic; exact McNemar $p = 3.2 \times 10^{-58}$). Against the static manifest there were 74 discordant cases, all favouring dynamic ($p = 1.1 \times 10^{-22}$). All three draft hypothesis thresholds were met; we stress again that this run is exploratory.

6.2 Blind spots by mutation family

Figure 2 breaks recall down by family. The pattern is sharp: each cheap check covers a fixed subset of families completely and the rest not at all. Caching, dependency and hidden-configuration failures were invisible to both non-executing checks.

6.3 Localisation and cost

Every method that detected a mutation also attributed it to the declared stage (seed-only 80/80, static 198/198, dynamic 272/272). For the static manifest this is partly by construction, since configuration sections map one-to-one onto stages in our design. For the dynamic auditor it is an empirical result: the first divergent fingerprint fell at the data stage in 142 cases, preprocess in 70, model in 36 and split in 24, matching the oracle in every case. Re-execution is the costliest method, with a median of 4.6 ms per audit (maximum 255 ms; 31.9 ms median for clustering experiments), roughly 50 times the static check. At this scale the absolute cost is negligible; for realistic training workloads it is not, which is why the cheaper checks matter.

6.4 Invariances found by the oracle

The first oracle pass disagreed with 25 of 476 initially declared cases. Each disagreement was a positive that did *not* change the result. We resolved them by narrowing operator applicability before the reported run, reducing the case count to 452, and record them here because they are informative in their own right:

1. **Label-sorted rows with a stratified split.** Sorting the input by target before a stratified train/test split did not change any classification result (16 cases). Stratified shuffling with a

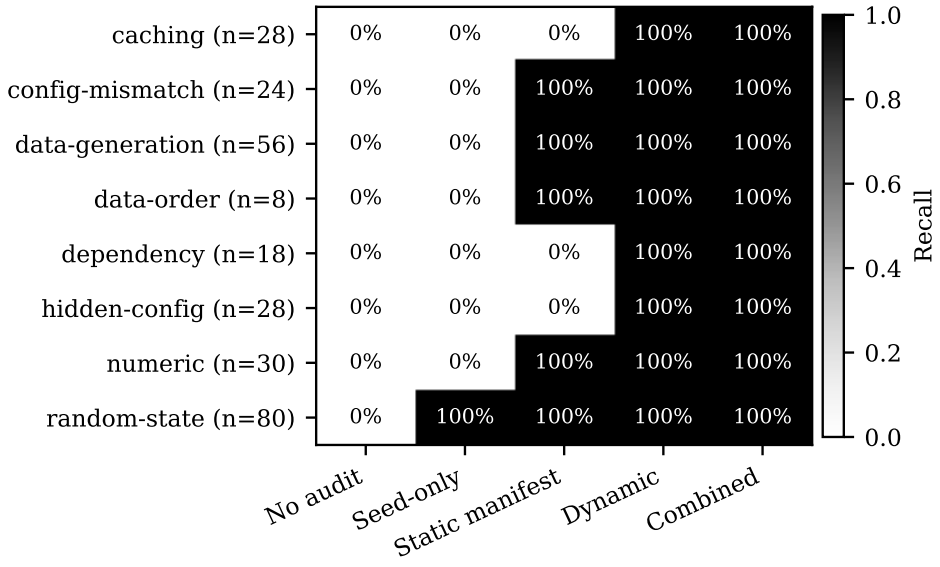


Figure 2: Recall by mutation family and method. n is the number of positive cases in the family.

stable sort selected the same rows, so data-stage artifacts differed while results did not. The operator was restricted to unstratified regression.

2. **float32 features with tree ensembles.** Reducing feature precision to float32 left random-forest results unchanged (6 cases). The operator was restricted to non-tree models.

3. **Loosened tolerance for a Gaussian mixture on separated clusters.** EM converged within one iteration, so loosening the tolerance changed nothing (1 case). A further 2 k-means cases first diverged at the metrics stage rather than the model stage; we added the clustering objective to the model-stage artifacts.

Items 1 and 2 are natural *hard controls*: intermediate artifacts change while the result does not. An auditor that compares intermediate artifacts too strictly would raise false alarms on them.

7 External validation on independently written code (E1)

The synthetic suite controls ground truth but was designed by the author. To test whether its conclusions transfer, we ran a second, *pre-registered* study on code the author did not write. The protocol (population, sampling seed, eligibility rules, operators, oracle and analysis) was committed and pushed to the public repository before any run (`PROTOCOL-E1-external.md`, commit `382e5fd`); all deviations are listed in `results/e1-sklearn-examples/DEVIATIONS.md`.

Population and sampling. All 279 `examples/` scripts of scikit-learn 1.7.2 (commit `25dee60`), written and reviewed by that project's contributors. Scripts were screened in a random order seeded from the v0.1 preprint DOI; the sample is the first 30 eligible scripts. A script was eligible if it needed no network download (E1), ran within 30 s (E2), printed at least one number, taken as its reported result after excluding timing lines (E3), and gave the same numbers on two baseline runs (E4).

Table 4: E1: 199 mutation cases on 30 independently written scripts. Truth is the execution oracle. Wilson 95% intervals.

Auditor	Registered (crash = changed) Recall	FPR	Sensitivity (crashes re-run) Recall	FPR
Seed-only	71.1% (60.0–80.0)	1.6% (0.4–5.7)	75.4% (64.0–84.0)	3.1% (1.2–7.6)
Static manifest	93.4% (85.5–97.2)	6.5% (3.3–12.3)	100% (94.7–100)	7.7% (4.2–13.6)
Dynamic ($k = 2$)	98.7% (92.9–99.8)	0.0% (0.0–3.0)	98.6% (92.2–99.7)	0.0% (0.0–2.9)
Result changed / cases	76 / 199		69 / 199	

Ground truth by execution. Unlike the synthetic suite, labels are not declared: operators state an *intent* and the oracle (two runs, full-precision tolerance comparison against both baselines and each other) decides whether the result changed. Source operators were applied through the abstract syntax tree: seeds removed or changed, train/test split size changed, fold count changed, iteration counts halved; controls were an added comment, an AST reformat, `OMP_NUM_THREADS=1` and a moved working directory.

Screening. Reaching 30 eligible scripts required screening 114: 27 needed downloads, 4 failed or timed out, 51 printed no numbers (their results are plots only), and **2 of the 32 runnable, number-printing scripts were not reproducible run-to-run before any mutation**.

Results. Table 4 reports both the registered analysis and a sensitivity analysis. Nine mutation units crashed only when scripts ran four at a time; re-run alone, none crashed (deviation 3), so we treat the sensitivity analysis as the better estimate while reporting both.

Four observations follow.

1. **The seed-only blind spot transfers.** It missed every changed split size and fold count (17 cases in the sensitivity analysis; dynamic vs seed-only, exact McNemar $p = 1.5 \times 10^{-4}$).
2. **Equivalent changes are common and cause static false alarms.** Of 79 positive-intent mutations, 10 (12.7%) did not change the reported result: both halved-iteration cases, 2 of 3 fold-count changes, 2 of 18 split-size changes, and 4 seed changes in scripts whose printed numbers do not depend on randomness. The static manifest flagged all 10.
3. **Static and dynamic detection did not differ significantly on real code** (sensitivity: 0 vs 1 discordant cases). This does not contradict the synthetic finding: none of the generic environment operators changed any result, so E1 did not exercise the hidden-state failures (stale caches, environment overrides, dependency drift) that drove the synthetic gap. That claim remains supported only by the synthetic suite.
4. **Re-execution has its own failure modes.** With $k = 2$ the dynamic auditor missed one seed removal whose effect appeared only in some runs (`plot_rfe_with_cross_validation.py`), and under parallel load a "crash means flag" policy produced nine spurious alarms that disappeared on isolated re-runs.

8 Discussion

What the results do and do not show. The dynamic auditor and the oracle both execute the package, so near-perfect dynamic recall on deterministic, single-fault mutations is expected

and should not be read as evidence of real-world effectiveness. The informative results concern the non-executing checks: their blind spots are structural and complete. A seed check cannot see anything but seeds, and a configuration manifest cannot see anything outside the configuration. The 74 hidden mutations are exactly the cases that a careful reader of a repository's configuration would also miss.

False alarms have a cost. Static comparison flagged every harmless thread-count change. In practice, false alarms train users to ignore a tool. A practical design is therefore layered: run cheap static checks always, and reserve re-execution, with at least two repeats to expose nondeterminism, for changes the static layer cannot clear or for pre-release audits.

Relation to studies on real repositories. Shulepov [12] found that existing validation workflows in real ML repositories detected few controlled reproducibility mutations. Our controlled results suggest one mechanism consistent with that observation: checks that do not compare re-executed results against a recorded reference are blind to whole families of change. E1 is a first step: on independently written scripts the seed-only blind spot reproduced, but the hidden-state mechanism could not be exercised with generic operators, so testing it on real research repositories remains future work.

9 Limitations and threats to validity

- **Circularity.** The dynamic auditor and the oracle share the principle of re-execution (see Section 8).
- **Scope of the evidence.** The synthetic suite uses 400-sample scikit-learn experiments on generated data. E1 adds 30 independently written scikit-learn example scripts, which are documentation code rather than research-paper code, and only scripts that print numbers. No GPU, distributed or deep-learning workloads and no research-paper repositories have yet been audited.
- **Author-designed taxonomy.** The operators were chosen by the author and have not been validated against an empirical sample of real reproducibility failures; recall depends directly on which operators are included.
- **Simulated dependency drift** through a stand-in library rather than real package upgrades.
- **Single faults only**; interactions between two or more faults are untested.
- **One execution environment** for the benchmark run; cross-platform agreement of the benchmark itself is untested (the unit tests passed on two operating systems).
- **Applicability was narrowed after the first oracle pass** (Section 6.4). The narrowing was driven by the oracle, not by auditor results, but it is a researcher degree of freedom and the run is therefore not confirmatory.
- **Fingerprint rounding.** Hashing values rounded to 8 decimals can disagree with a tolerance comparison at rounding boundaries; no such disagreement occurred in this run.
- **Reproduction is not correctness.** Passing an audit does not imply that an experiment is scientifically valid.

10 Planned work

Before v0.1.0 we plan to: (i) publicly preregister the hypothesis and analysis plan and run the frozen benchmark as the confirmatory study; (ii) add two-fault interactions and the hard controls of Section 6.4; (iii) run the benchmark across operating systems and library versions; and (iv) conduct a systematic literature review. Stage E2 will apply the operators to an independently sampled set of research-paper repositories (for example, those of ML Reproducibility Challenge reports), executed only in isolated, ephemeral CI runners after human review of the sampled list, under a separately pre-registered protocol. It should add hidden-state operators that work on real code, such as dependency-version substitution, and a result definition that covers plotted outputs.

11 Availability

Source code, experiment generators, mutation operators, auditors, oracle, tests and the raw case-level results (`results/dev-2026-11-08/cases.jsonl`, SHA-256 `a33dcec9...2828`; E1: `results/e1-sklearn-examples/`) are available at https://github.com/Hamjeth68/repromut. Reproduce with `pip install -e ".[test]" && pytest -q && repromut bench -out results/full`; a full run takes under one minute on a laptop CPU. Code is released under the Apache License 2.0, documentation under CC BY 4.0 and generated fixtures under CC0.

Declarations

AI assistance. Substantial parts of the software and of this manuscript were produced with an AI assistant (Claude, Anthropic) under the author's direction. All numerical results are produced by the released scripts, and the author takes responsibility for the content.

Funding. None.

Competing interests. The author declares no competing interests.

Data. All data are synthetic and generated by the released code; no human-subject or personal data are involved.

References

[1] Xavier Bouthillier et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3, 2021.

[2] Fernando Chirigati, Rémi Rampin, Dennis Shasha, and Juliana Freire. ReproZip: Computational reproducibility with ease. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*, 2016. doi: 10.1145/2882903.2899401.

[3] Saikat Dutta, Anshul Arunachalam, and Sasa Misailovic. To seed or not to seed? an empirical analysis of usage of seeds for testing in machine learning projects. In *Proceedings of the 15th IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2022.

[4] Odd Erik Gundersen and Sigbjørn Kjensmo. State of the art: Reproducibility in artificial intelligence. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, pages 1644–1651, 2018. doi: 10.1609/aaai.v32i1.11503.

[5] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

[6] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023. doi: 10.1016/j.patter.2023.100804.

[7] Lei Ma, Fuyuan Zhang, Jiyuan Sun, Minhui Xue, Bo Li, Felix Juefei-Xu, Chao Xie, Li Li, Yang Liu, Jianjun Zhao, and Yadong Wang. DeepMutation: Mutation testing of deep learning systems. In *Proceedings of the 29th IEEE International Symposium on Software Reliability Engineering (ISSRE)*, pages 100–111, 2018. doi: 10.1109/ISSRE.2018.00021.

[8] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[9] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[10] João Felipe Pimentel, Leonardo Murta, Vanessa Braganholo, and Juliana Freire. A large-scale study about quality and reproducibility of Jupyter notebooks. In *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*, pages 507–517, 2019.

[11] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22:1–20, 2021.

[12] Ilya Shulepov. Mutation testing for reproducibility safeguards in machine learning research software: An empirical study, 2026.

[13] Ana Trisovic, Matthew K. Lau, Thomas Pasquier, and Mercè Crosas. A large-scale study on research code quality and execution. *Scientific Data*, 9:60, 2022. doi: 10.1038/s41597-022-01143-6.

[14] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[15] Xiaoyuan Xie, Joshua W. K. Ho, Christian Murphy, Gail Kaiser, Baowen Xu, and Tsong Yueh Chen. Testing and validating machine learning classifiers by metamorphic testing. *Journal of Systems and Software*, 84(4):544–558, 2011. doi: 10.1016/j.jss.2010.11.920.

[16] Donglin Zhuang, Xingyao Zhang, Shuaiwen Song, and Sara Hooker. Randomness in neural network training: Characterizing the impact of tooling. In *Proceedings of Machine Learning and Systems*, volume 4, 2022.